

University Of Engineering and Technology, Lahore
Computer Engineering Department

Course Name: Fundamentals of Programming & Data Science	Course Code: CMPE-112L
Assignment Type: Lab Report	Dated: July 1, 2026
Semester: 1st Semester	Session: Spring 2026
Lab/Project/Assignment #: 12	CLOs to be covered: CLO2
Lab Title: Lambda Functions	Teacher Name: Hafiz Muhammad Mubashar
Student Name / Roll No: Hashir Zahid (2026(S)-CYS-40)	

Lab Evaluation:

CLO2	Design and implement modular programs using functions, recursion, and reusable code constructs.					
	Level1	Level2	Level3	Level4	Level5	Level6
(5)						
Total						/5

Rubrics for Current Lab Evaluation

Scale	Marks	Level	Rubric
Excellent	5	L1	Submitted all lab tasks, BONUS task, have good

			understanding.
Very Good	4	L2	Submitted the lab tasks but have good understanding
Good	3	L3	Submitted the lab tasks but have weak understanding.
Basic	2	L4	Submitted the lab tasks but have no understanding.
Barely Acceptable	1	L5	Submitted only one lab task.
Not Acceptable	0	L6	Did not attempt

LAB No 12

Lab Goals / Objectives:

- Understand the concept of lambda (anonymous) functions
- Learn the syntax and structure of lambda functions
- Differentiate between lambda functions and regular functions
- Use lambda functions with built-in functions like map(), filter(), and sorted()
- Apply lambda functions to practical programming problems
- Combine lambda with conditional expressions
- Understand when to use lambda vs regular functions

Equipment Required:

- Computer System with Python 3.x installed
- Text Editor or IDE (PyCharm, VS Code, or IDLE)
- Operating System: Windows / Linux / macOS

Theory:

1. What are Lambda Functions?

Lambda functions (also called anonymous functions) are small, restricted functions that are not bound to an identifier. They are defined using the lambda keyword instead of def. Lambda functions can have any number of arguments but only one expression.

2. Lambda Syntax

The syntax of a lambda function is:

```
lambda arguments : expression
```

The expression is evaluated and returned when the lambda is called. Lambda functions can be assigned to variables or used inline.

3. Lambda vs Regular Functions

Feature	Lambda	Regular Function
Syntax	lambda args: expr	def name(args): return expr
Name	Anonymous (no name)	Has a name
Body	Single expression	Multiple statements
Return	Implicit return	Explicit return statement
Use	Short, simple operations	Complex logic

4. Lambda with Multiple Arguments

Lambda functions can accept multiple arguments separated by commas:

```
# Lambda with two arguments
multiply = lambda x, y: x * y
print(multiply(5, 3)) # Output: 15
```

5. Lambda with Conditional Expressions

Lambda functions can include conditional expressions for simple decision-making:

```
# Lambda to find maximum
maximum = lambda a, b: a if a > b else b
print(maximum(10, 20)) # Output: 20
```

6. The map() Function

The map() function applies a given function to every item of an iterable and returns an iterator. It is commonly used with lambda:

```
numbers = [1, 2, 3, 4, 5]
squares = list(map(lambda x: x**2, numbers))
print(squares) # [1, 4, 9, 16, 25]
```

7. The filter() Function

The filter() function constructs an iterator from elements of an iterable for which a function returns True:

```
numbers = [1, 2, 3, 4, 5, 6]
evens = list(filter(lambda x: x % 2 == 0, numbers))
print(evens) # [2, 4, 6]
```

8. Using Lambda with sorted()

The sorted() function accepts a key parameter that can be a lambda function for custom sorting:

```
students = [('Ali', 85), ('Ahmed', 92), ('Bilal', 78)]
sorted_by_score = sorted(students, key=lambda x: x[1])
```

9. When to Use Lambda Functions

- For short, throwaway functions that are used once
- As arguments to higher-order functions (map, filter, sorted)
- For simple transformations that fit in one expression
- When defining a full function would be unnecessarily verbose

10. When NOT to Use Lambda Functions

- For complex logic requiring multiple statements
- When the function needs to be reused extensively
- When readability would be compromised
- For functions that need documentation (docstrings)

Lab Work:

Task 1: Greet Function with Name and Age

Write a program that defines a function to greet a person by name and display their age.

Code Summary:

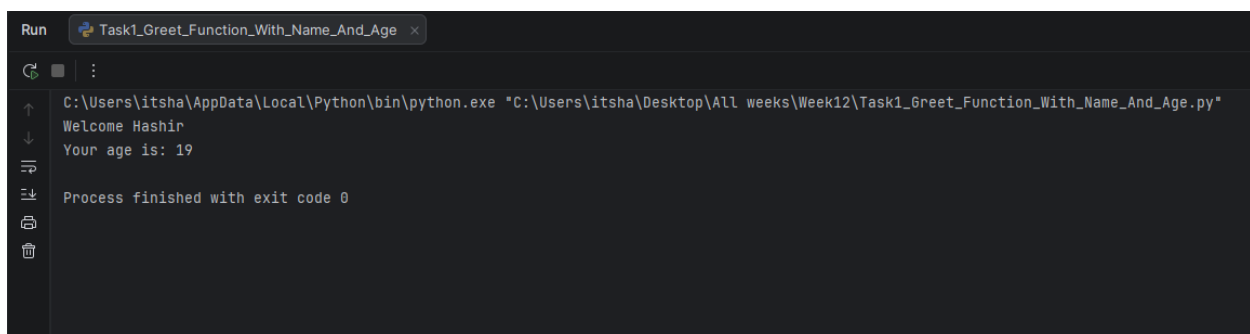
This task reviews regular function definition with multiple parameters as a foundation for comparing with lambda functions.

Key Code Lines:

```
# Greet Function with Name and Age
def greet(name, age):
    print("Welcome", name)
    print("Your age is:", age)

greet("Hashir", 19)
```

Output:



```
Run Task1_Greet_Function_With_Name_And_Age x
C:\Users\itsha\AppData\Local\Python\bin\python.exe "C:\Users\itsha\Desktop\All weeks\Week12\Task1_6reet_Function_With_Name_And_Age.py"
Welcome Hashir
Your age is: 19
Process finished with exit code 0
```

Task 2: Lambda Square Function

Write a lambda function that calculates the square of a number.

Code Summary:

This task demonstrates the basic lambda syntax. The lambda takes one argument and returns its square, achieving in one line what would normally require a full function definition.

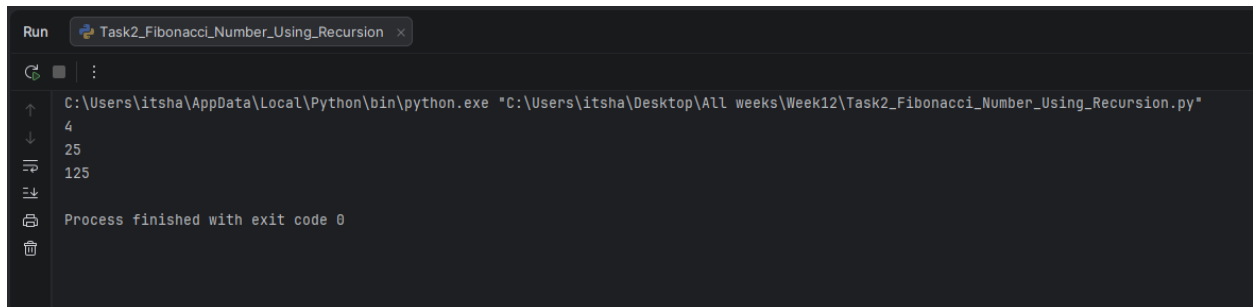
Key Code Lines:

```
# Lambda Square Function
square = lambda number: number ** 2
print(square(2))
```

```
# Lambda with two arguments
multiply = lambda x, y: x * y
print(multiply(5, 5))
```

```
# Lambda with three arguments
product = lambda x, y, z: x * y * z
print(product(5, 5, 5))
```

Output:



```
Run Task2_Fibonacci_Number_Using_Recursion x
C:\Users\itsha\AppData\Local\Python\bin\python.exe "C:\Users\itsha\Desktop\All weeks\Week12\Task2_Fibonacci_Number_Using_Recursion.py"
4
25
125
Process finished with exit code 0
```

Task 3: Lambda with map() Function

Use lambda with the map() function to square all elements in a list.

Code Summary:

This task demonstrates combining lambda with map() to apply an operation to every element of a list efficiently without writing an explicit loop.

Key Code Lines:

```
# Lambda with map()
x = [1, 2, 3, 4, 5]
```

```
squared = list(map(lambda x: x * x, x))
print(squared)
```

Output:

```
Run Task2_Fibonacci_Number_Using_Recursion x
C:\Users\itsha\AppData\Local\Python\bin\python.exe "C:\Users\itsha\Desktop\All weeks\Week12\Task2_Fibonacci_Number_Using_Recursion.py"
[1, 4, 9, 16, 25]
Process finished with exit code 0
```

Task 4: Lambda Maximum with Conditional

Write a lambda function that returns the maximum of two numbers using a conditional expression.

Code Summary:

This task demonstrates using a conditional expression within a lambda to make decisions. The syntax 'a if condition else b' evaluates to a if the condition is True, otherwise b.

Key Code Lines:

```
# Lambda Maximum with Conditional
maximum = lambda a, b: a if a > b else b
print("Max of 10, 20:", maximum(10, 20))
print("Max of 50, 30:", maximum(50, 30))
```

Output:

```
Run task1 x
C:\Users\itsha\AppData\Local\Python\bin\python.exe "C:\Users\itsha\Desktop\All weeks\Week11\task1.py"
Max of 10, 20: 20
Max of 50, 30: 50
Process finished with exit code 0
```

Task 5: Lambda with filter() Function

Use lambda with the filter() function to extract even numbers from a list.

Code Summary:

This task shows how lambda combined with filter() creates a concise way to select elements from a list that satisfy a condition.

Key Code Lines:

```
# Lambda with filter()
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
evens = list(filter(lambda x: x % 2 == 0, numbers))
print("Even numbers:", evens)
```

Output:



```
Run task1 x
C:\Users\itsha\AppData\Local\Python\bin\python.exe "C:\Users\itsha\Desktop\All weeks\Week11\task1.py"
Even numbers: [2, 4, 6, 8, 10]
Process finished with exit code 0
```

Conclusions:

- Successfully understood the concept and syntax of lambda (anonymous) functions
- Differentiated between lambda functions and regular named functions
- Learned to use lambda with multiple arguments and conditional expressions
- Applied lambda with map() to transform all elements of a list
- Applied lambda with filter() to select elements based on a condition
- Understood the appropriate use cases for lambda functions
- Gained the ability to write more concise and functional Python code